



University of San Carlos | Department of
COMPUTER ENGINEERING

CpE 3104 – Microprocessors

Hardware Interrupt Interfacing

UNIT 8: INTERRUPT INTERFACING

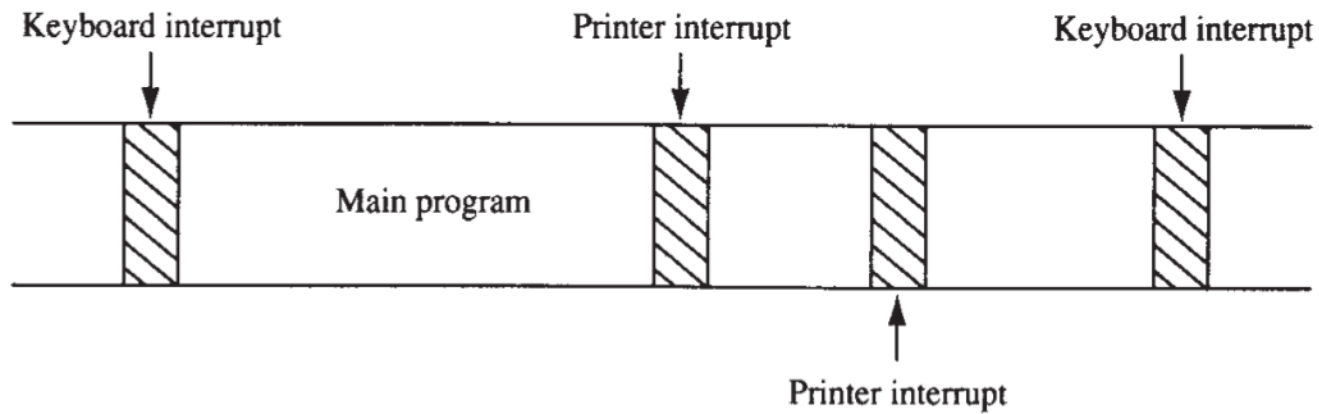
© 2025

Contents

- Why Interrupts?
- Interrupts in 8086 Microprocessor
- Hardware Interrupts
- Introduction to 8259 Programmable Interrupt Controller

Purpose of Interrupts

- Interrupts are particularly useful when interfacing I/O devices that provide or require data at relatively low data transfer rates.
- This is to prevent the CPU being delayed by I/O devices since the data transfer rate is much, much slower than the CPU.
- It allows the CPU to perform other tasks than waiting for an I/O data.



A timeline that indicates interrupt usage in a typical system.

Interrupting the CPU

- While the CPU is performing foreground, it is interrupted by I/O devices in which the CPU will leave the execution of the foreground routine and execute a program to handle the interrupt.
- This is an efficient solution since I/O devices are mostly non-periodic such as a keyboard or printer.
- It also ensures efficient utilization of the CPU.

Hardware Interrupts (Intel)

- The interrupts of the entire Intel family of microprocessors include two hardware pins that request interrupts (**INTR** and **NMI**), and one hardware pin (**$\overline{INTA}$**) that acknowledges the interrupt requested through **INTR**.

Interrupt Vectors

- The interrupt vector is a pointer to the address of its interrupt handler or the code to be executed by the CPU to a particular interrupt request.
- The interrupt vector table is located in the first 1024 bytes of memory at addresses 00000H–003FFH. It contains **256** different **four-byte** interrupt vectors.
- An interrupt vector contains the address (**segment** and **offset**) of the interrupt service procedure.

	Type 32 — 255 User interrupt vectors
080H	Type 14 — 31 Reserved
040H	Type 16 Coprocessor error
03CH	Type 15 Unassigned
038H	Type 14 Page fault
034H	Type 13 General protection
030H	Type 12 Stack segment overrun
02CH	Type 11 Segment not present
028H	Type 10 Invalid task state segment
024H	Type 9 Coprocessor segment overrun
020H	Type 8 Double fault
01CH	Type 7 Coprocessor not available
018H	Type 6 Undefined opcode
014H	Type 5 BOUND
010H	Type 4 Overflow (INTO)
00CH	Type 3 1-byte breakpoint
008H	Type 2 NMI pin
004H	Type 1 Single-step
000H	Type 0 Divide error

(a)

Any interrupt vector	
3	Segment (high)
2	Segment (low)
1	Offset (high)
0	Offset (low)

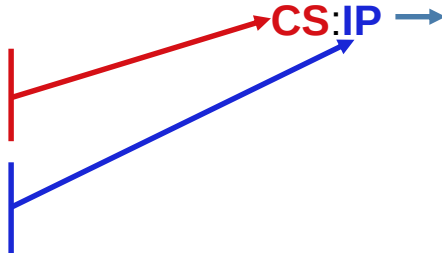
(b)

- (a) The **interrupt vector table** for the microprocessor and
(b) **Contents** of an interrupt vector

Interrupt Vectors

Interrupt Vector Table

...	
...	
00207H	segment high (81H)
00206H	segment low (81H)
00205H	offset high (81H)
00204H	offset low (81H)
00203H	segment high (80H)
00202H	segment low (80H)
00201H	offset high (80H)
00200H	offset low (80H)
...	
...	



...

...

0102AH

01029H

01028H

...

01003H

01002H

01001H

01000H

...

...

Interrupt Service Routine for Interrupt Vector 80H

Interrupt Instructions

- **BOUND:**
 - compares a register with two words of memory data.
- **INTO:**
 - checks or tests the **overflow flag (OF)**
- **INT:**
 - instruction calls the interrupt service procedure that begins at the address represented in vector number ***n***
- **INT3:**
 - is often used as a breakpoint-interrupt
- **IRET:**
 - is a special return instruction used to return for both software and hardware interrupts

Operation of a Real-Mode Interrupt

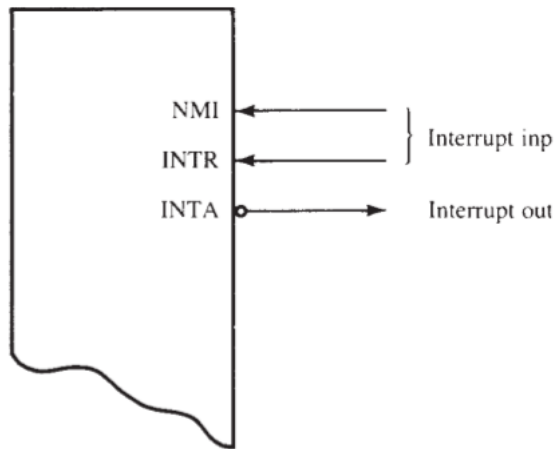
1. The contents of the flag register are pushed onto the stack.
2. Both the interrupt (IF) and trap (TF) flags are cleared. This disables the INTR pin and the trap or single-step feature.
3. The contents of the code segment register (CS) are pushed onto the stack.
4. The contents of the instruction pointer (IP) are pushed onto the stack.
5. The interrupt vector contents are fetched, and then placed into both IP and CS so that the
6. Next instruction executes at the interrupt service procedure addressed by the vector.

Operation of a Protected-Mode Interrupt

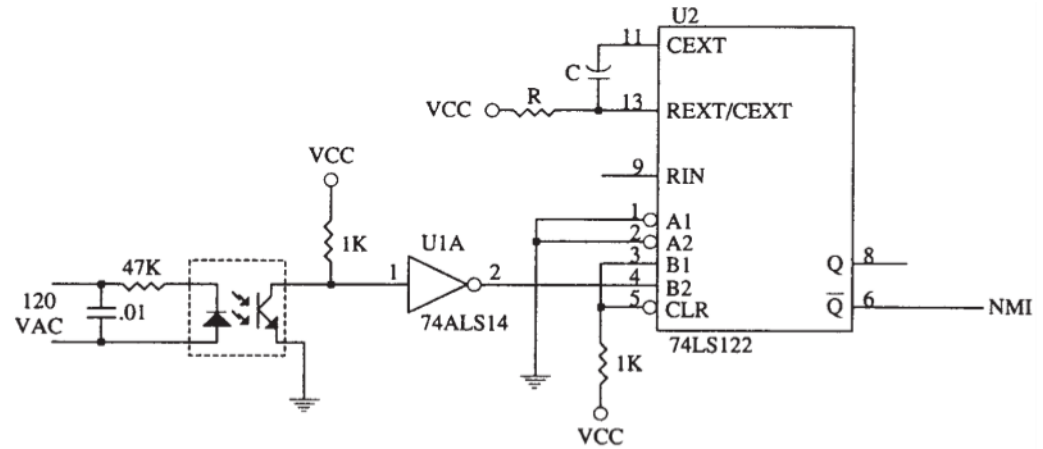
- In the protected mode, interrupts have exactly the same assignments as in the real mode, but the interrupt vector table is different.
- In place of interrupt vectors, protected mode uses a set of 256 interrupt descriptors that are stored in an **interrupt descriptor table (IDT)**.

Hardware Interrupts

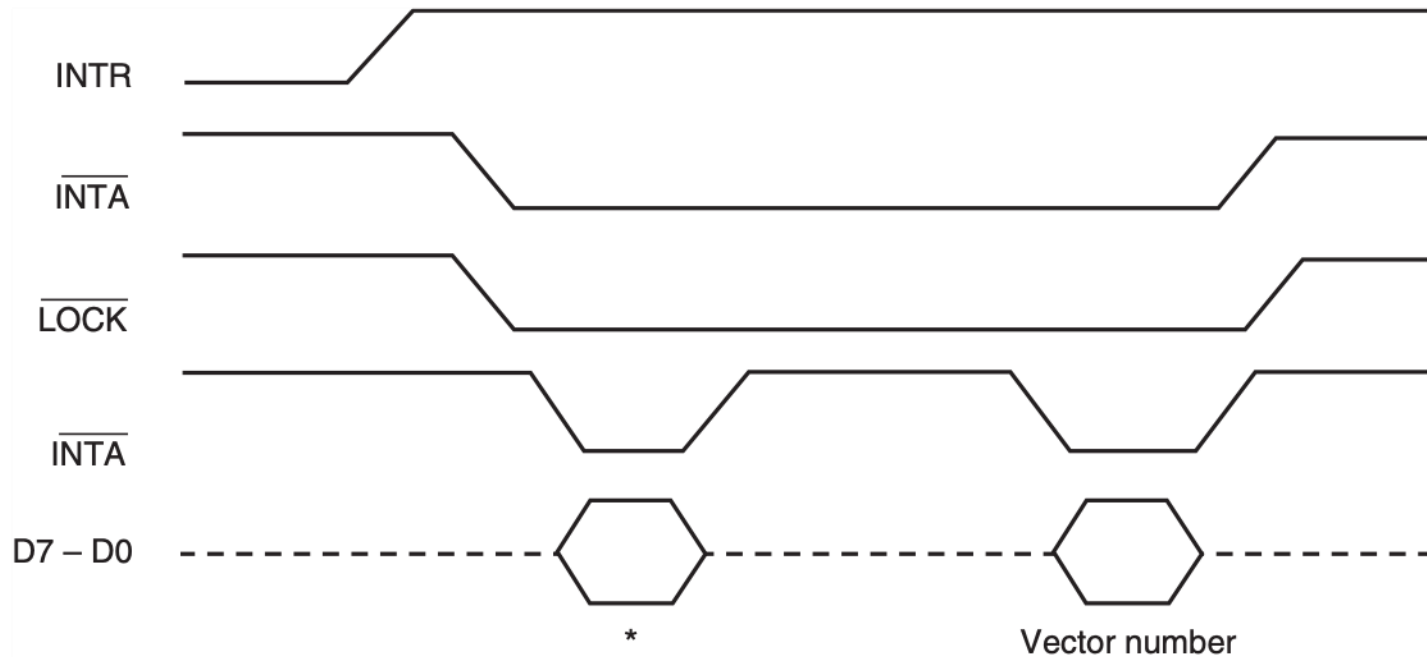
- The microprocessor has two hardware interrupt inputs: *non-maskable interrupt (NMI)* and *interrupt request (INTR)*.
- Whenever the **NMI input is activated**, a *Type 2 interrupt occurs* because **NMI is internally decoded**.
- The **INTR input** must be **externally decoded** to select a **vector**.
- The **$\overline{INTA}$** signal is also an interrupt pin on the microprocessor, but it is an **output** that is **used in response to the INTR input**.



The interrupt pins on all versions of the Intel microprocessor.



A power failure detection circuit which provides an interrupt signal via NMI.

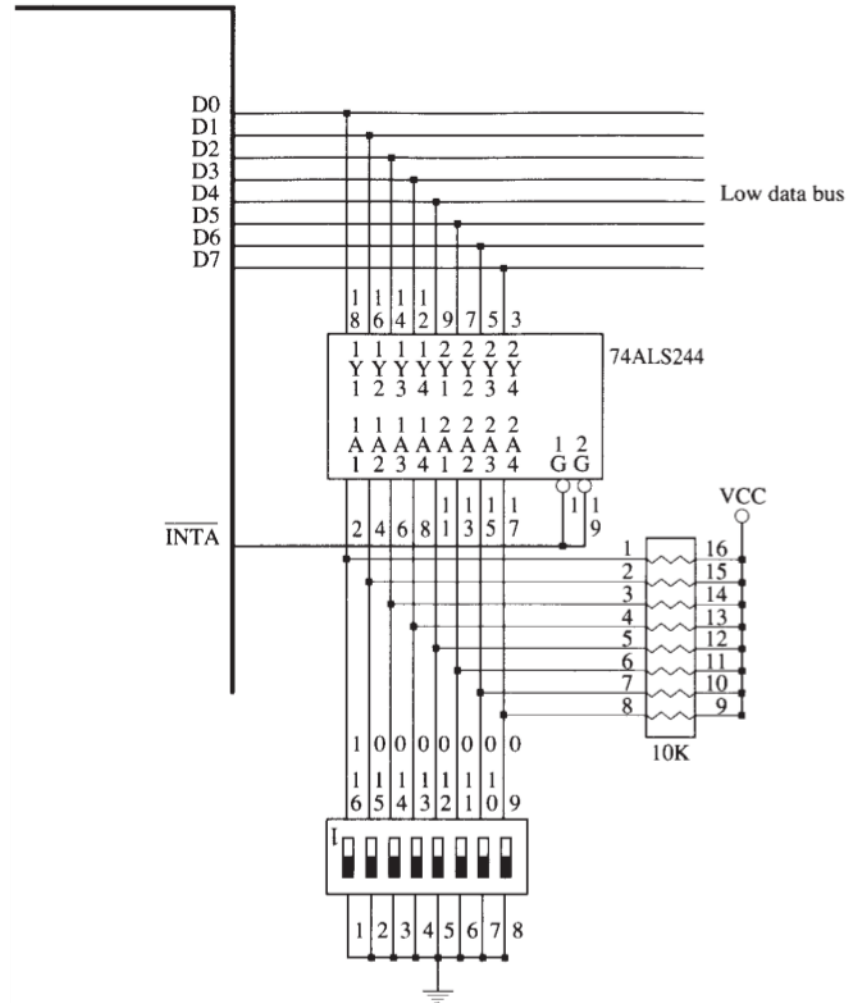


The timing of the INTR input and INTA output.

* This portion of the data bus is ignored and usually contains the vector number.

INTR and $\overline{\text{INTA}}$

- The interrupt request input (**INTR**) is level-sensitive, which means that it must be held at a **logic-1** level until it is recognized.
- The INTR pin is set by an external event and cleared inside the interrupt service procedure.
- The microprocessor responds to the INTR input by pulsing the $\overline{\text{INTA}}$ output in anticipation of receiving an interrupt vector type number on data bus connections D0–D7.

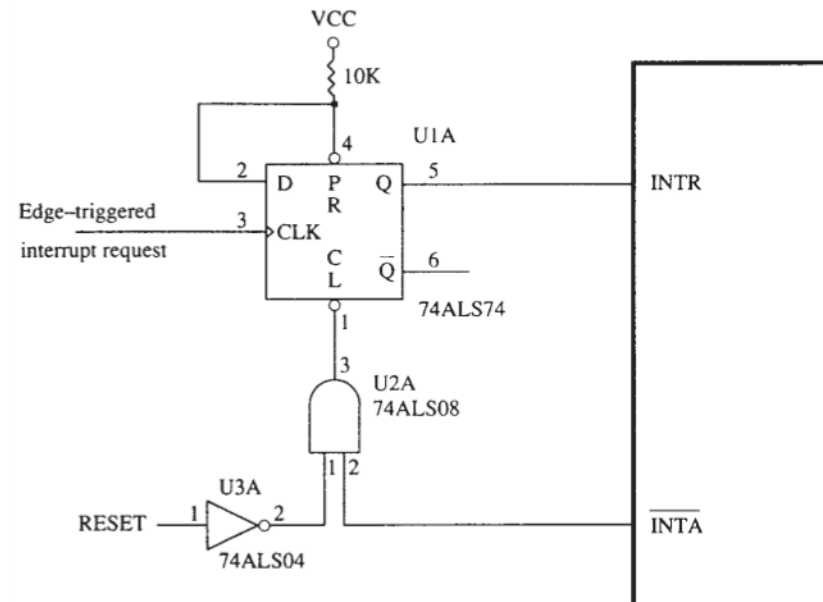


A circuit that applies any interrupt vector type number in response to INTA.

Here the circuit is applying type number 80H.

Other Methods of INTA Response

- Using a Three-State Buffer for INTA
- Making the INTR Input Edge-Triggered

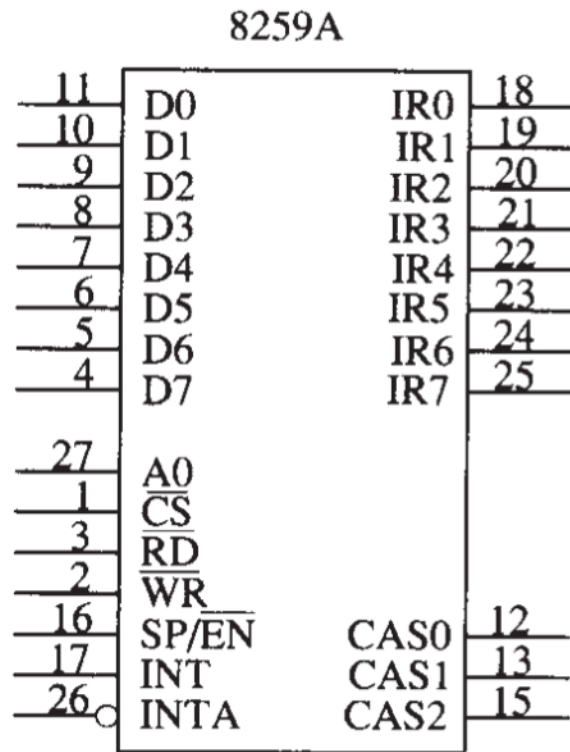


Converting INTR into an edge-triggered interrupt request input.

Intel 8259A Programmable Interrupt Controller

- The 8259A Programmable Interrupt Controller (PIC) adds eight vectored priority encoded interrupts to the microprocessor.
- This controller can be expanded, without additional hardware, to accept up to 64 interrupt requests.
- This expansion requires a master 8259A and eight 8259A slaves.

Intel 8259A Programmable Interrupt Controller



D0-D7 - Data Bus

IR7-IR0 - Interrupt Request Signals

$\overline{WR}$ - logic 0 when write operation

$\overline{RD}$ - logic 0 when read operation

A0 - command word select

CS - chip enable

INT - interrupt request to μP

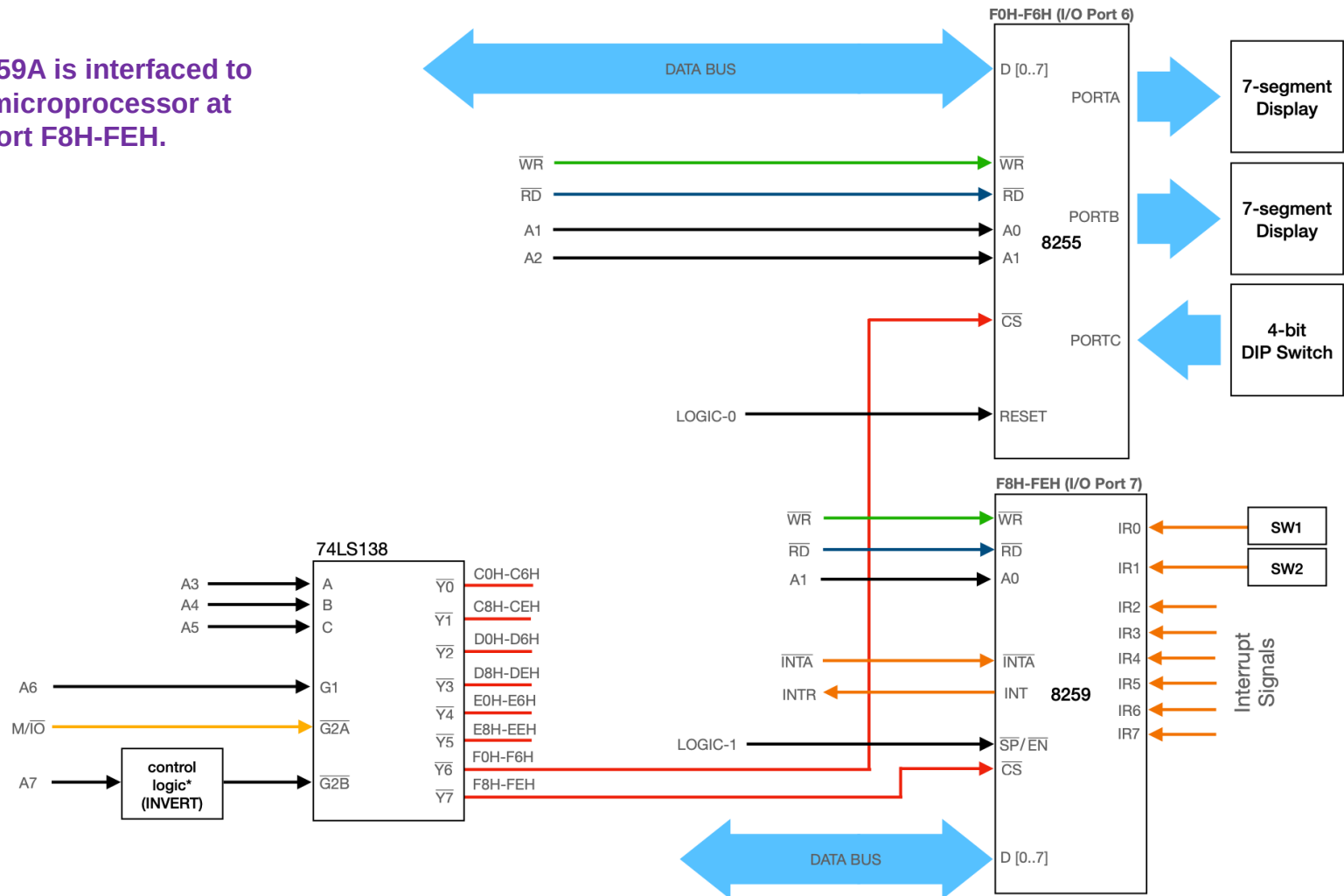
INTA - interrupt response from μP

$SP/\overline{EN}$ - slave program/enable buffer

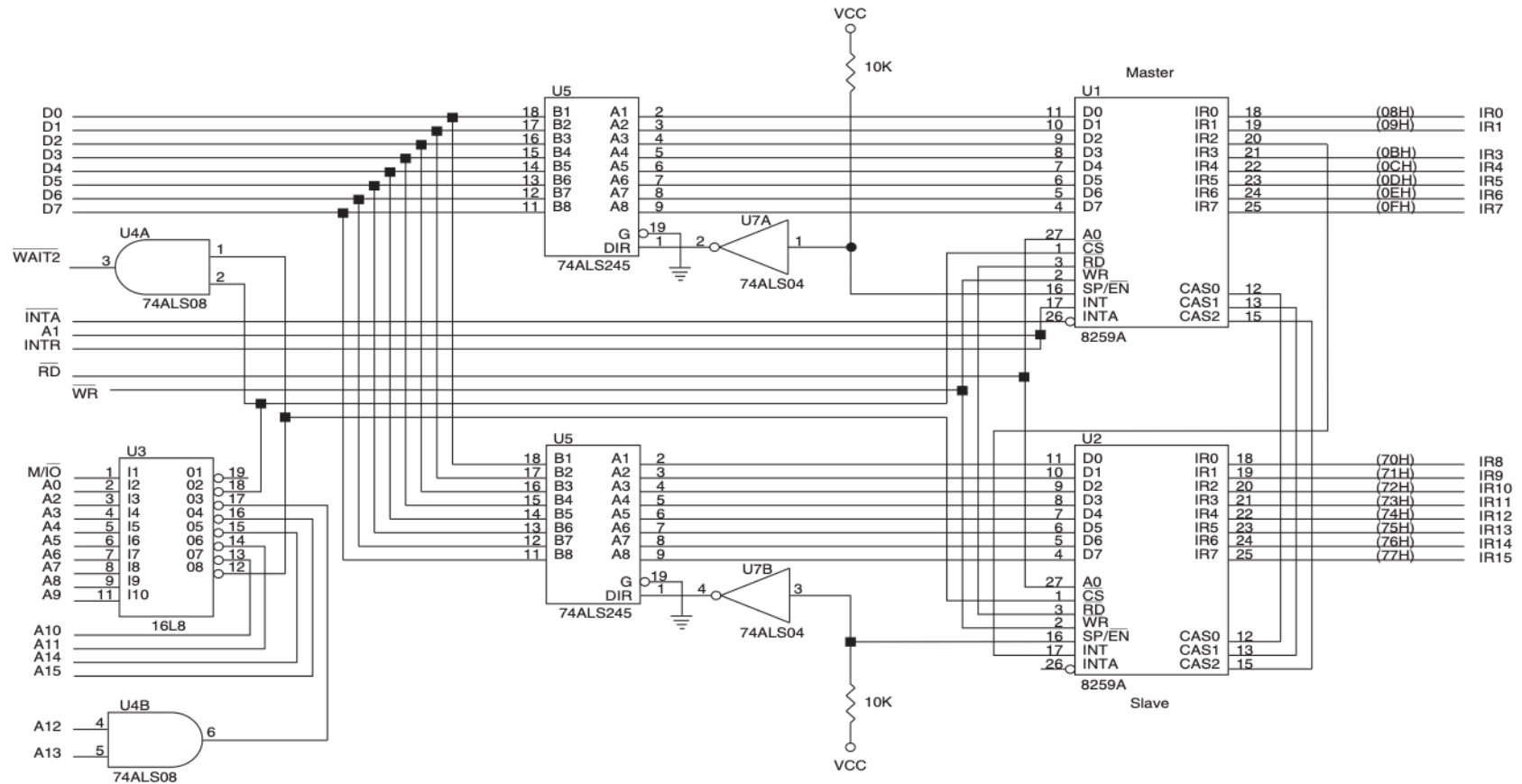
CAS0-CAS2 - cascade lines

Connecting a Single 8259

A single 8259A is interfaced to the 8086 microprocessor at I/O port F8H-FEH.



Connecting Cascading 8259As

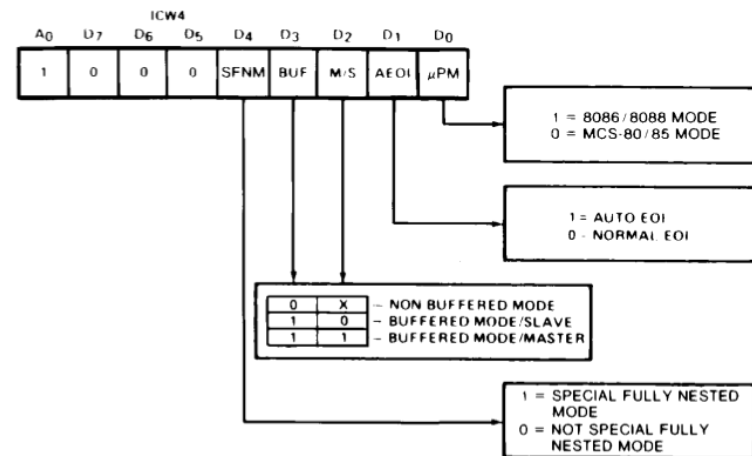
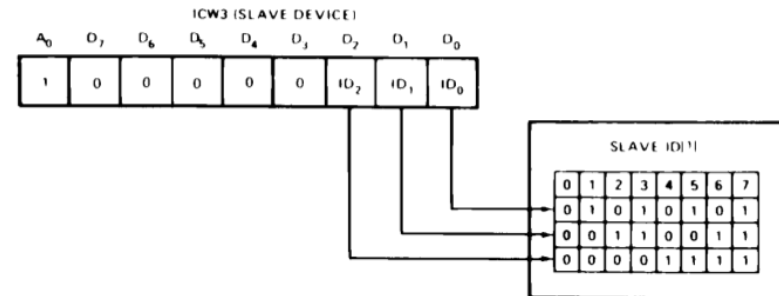
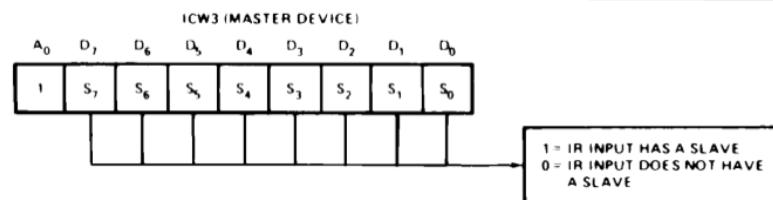
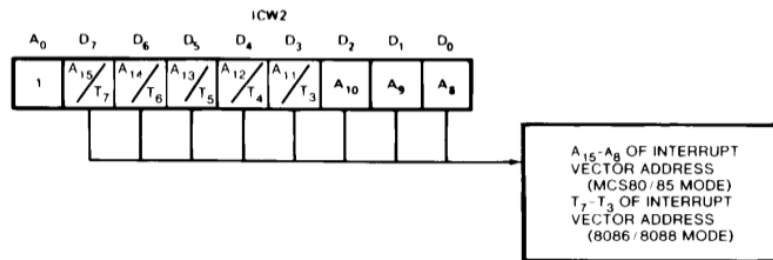
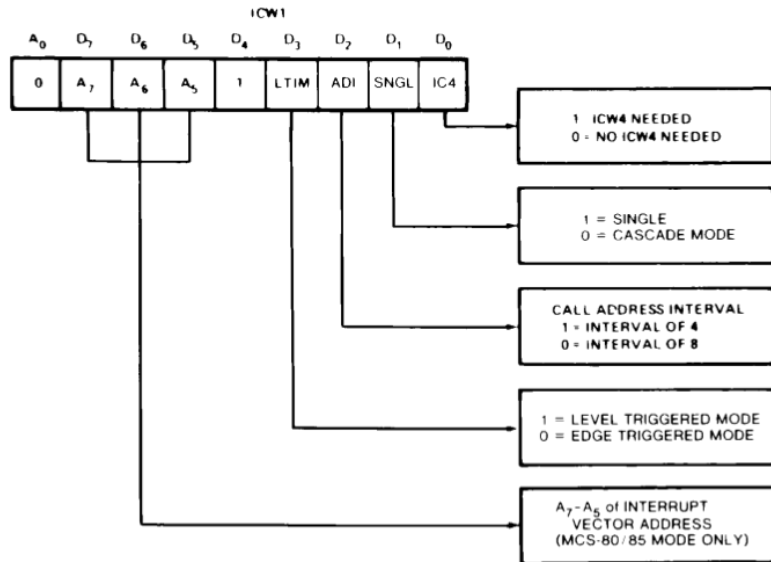


Two 8259As interfaced to the 8259A at I/O ports 0300H and 0302H for the master and 0304H and 0306H for the slave.

Programming the 8259A

- The 8259A is programmed by initialization and operation command words. Initialization command words (ICWs) are programmed before the 8259A is able to function in the system and dictate the basic operation of the 8259A.
- Operation command words (OCWs) are programmed during the normal course of operation.
- The OCWs control the operation of the 8259A.

Initialization Command Word (ICW) Format

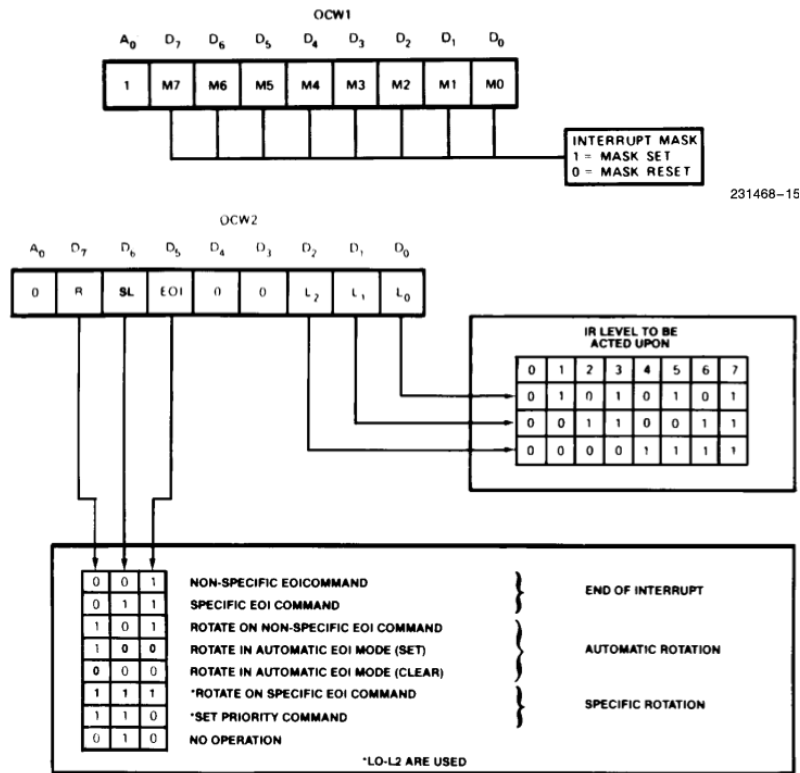


* A0 determines which ICW can be accessed

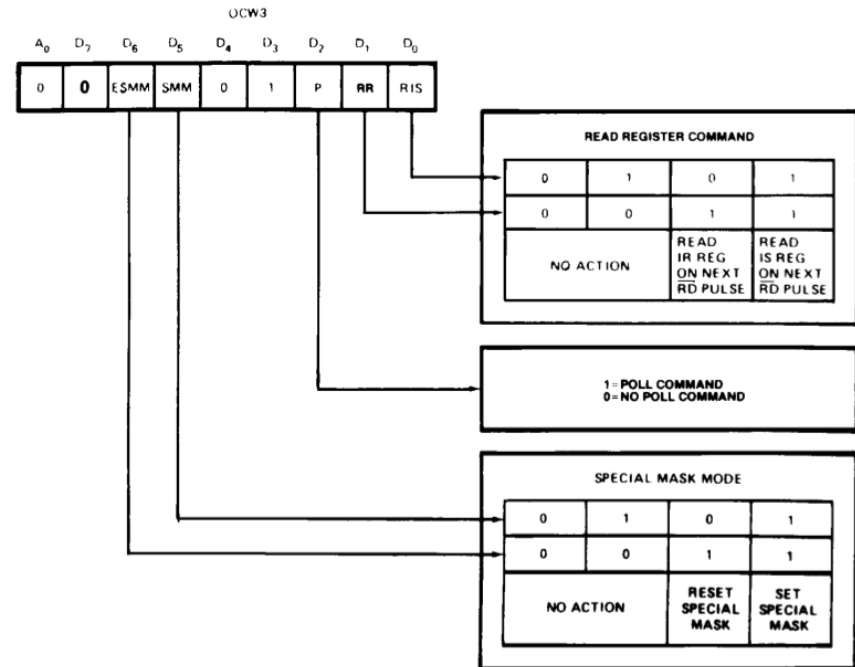
Initialization Command Words

- **ICW₁**: Programs the basic operation of the 8259A.
- **ICW₂**: Selects the vector number used with the interrupt request inputs.
- **ICW₃**: Only used when **ICW₁** indicates that the system is operated in cascade mode. This **ICW** indicates where the slave is connected to the master.
- **ICW₄**: Programmed for use with the 8086–Pentium 4 microprocessors, but is not programmed in a system that functions with the 8085 microprocessor.

Operation Command Word (OCW) Format



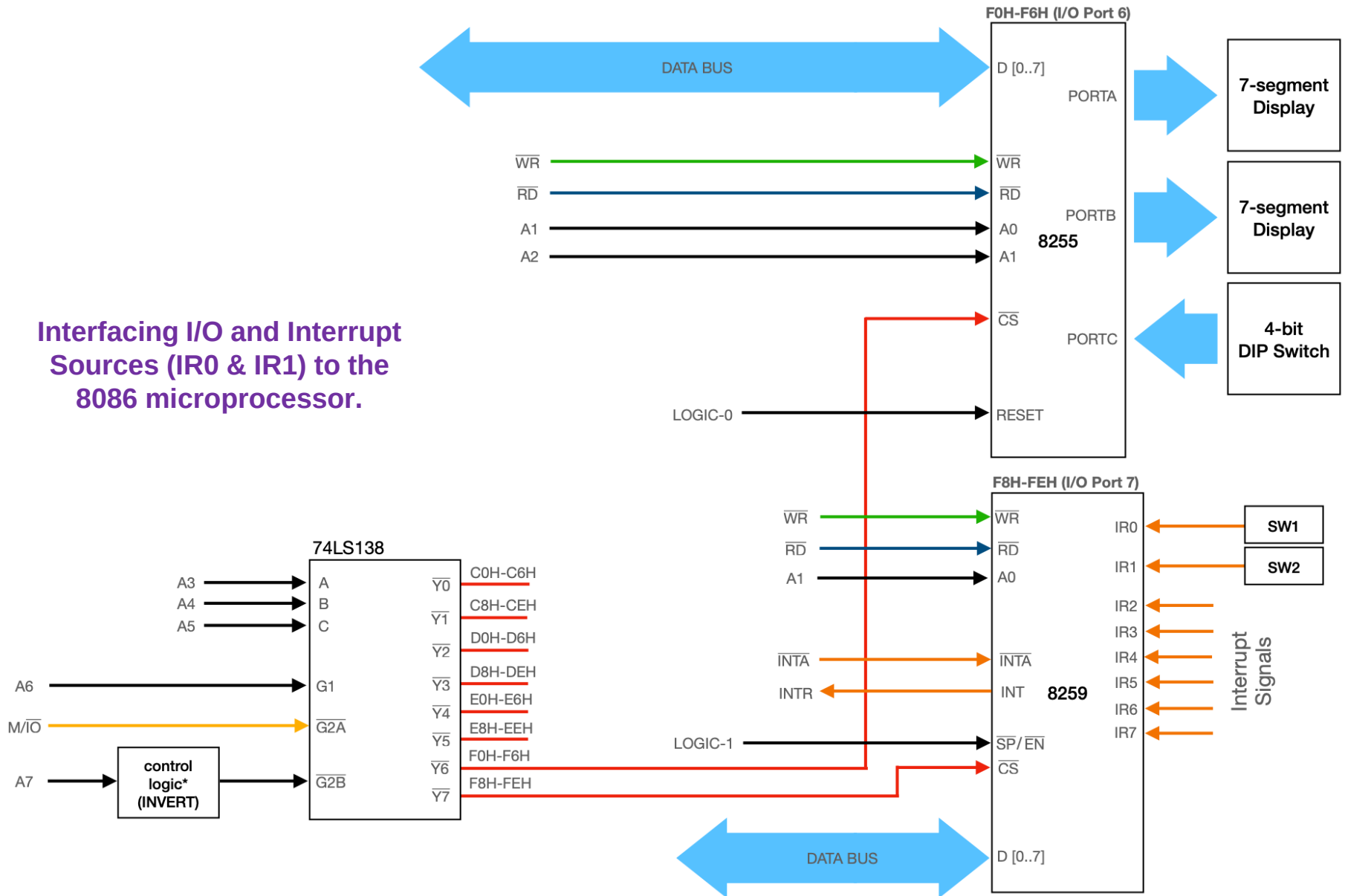
231468-15



Operation Command Words

- **OCW₁**: Used to set and read the interrupt mask register. When a mask bit is set, it will turn off (mask) the corresponding interrupt input.
- **OCW₂**: Programmed only when the AEOI mode is not selected for the 8259A.
- **OCW₃**: Selects the register to be read, the operation of the special mask register, and the poll command. If polling is selected, the P bit must be set and then output to the 8259A.

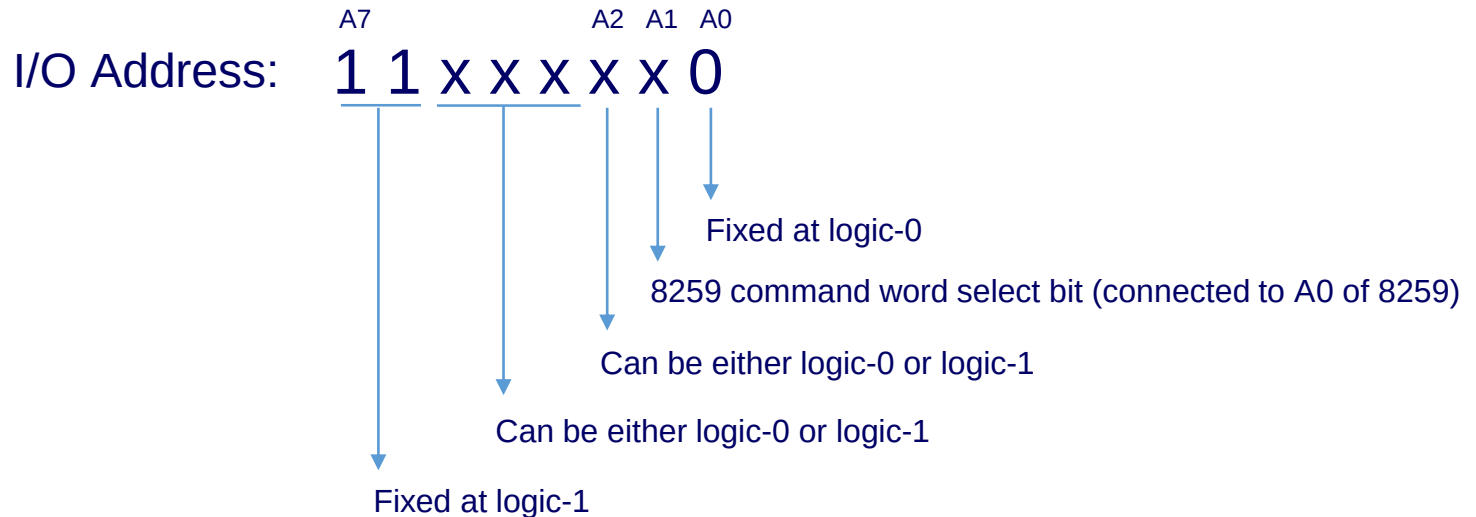
Interfacing I/O and Interrupt Sources (IR0 & IR1) to the 8086 microprocessor.



8259 Address Decoding

- The 8259 has two command words to be accessed when programming it: **ICW** and **OCW**.
- These command words can be accessed depending on the signal of **A0** of the 8259.
- In this example, we will decode the address **F8H-FEH** which will enable the 8259 (via the $\overline{CS}$ pin) as well as access ICWs or OCWs.

8259 Address Decoding



Possible addresses we
can use for accessing
the command words.

1 1 1 1 1 0 0 0	(F8H)	$A0 = 0$
1 1 1 1 1 0 1 0	(FAH)	$A0 = 1$
1 1 1 1 1 1 0 0	(FCH)	$A0 = 0$
1 1 1 1 1 1 1 0	(FEH)	$A0 = 1$

8259 Interrupt Vector Address

- The interrupt vector of the 8259 is controlled by ICW2.

T7	T6	T5	T4	T3	A10	A9	A8
1	0	0	0	0	0	0	0
					0	0	1
					0	1	0
					0	1	1
					1	0	0
					1	0	1
					1	1	0
					1	1	1

In this example, the interrupt vector address range is 80H-87H (IR0-IR7).

8259 Interrupt Vector Address

T7	T6	T5	T4	T3	A10	A9	A8
0	1	1	1	0	0	0	0
					0	0	1
					0	1	0
					0	1	1
					1	0	0
					1	0	1
					1	1	0
					1	1	1

In this example, the interrupt vector address range is 70H-77H (IR0-IR7).

8259 ICW and OCW

- In this example, we will program the 8259 as:
 - ICW₄ needed, single mode, call address interval of 8, edge triggered
 - 8259 Interrupt Vector addresses: 80H - 87H
 - 8086 mode, auto EOI
 - unmask only IR0 and IR1

DATA SEGMENT

```

    ORG 03000H           ; address in memory for this segment
    PORTA EQU 0F0H       ; 8255 address for PORTA
    PORTB EQU 0F2H       ; 8255 address for PORTB
    PORTC EQU 0F4H       ; 8255 address for PORTC
    COM_REG EQU 0F6H     ; 8255 address for command register
    PIC1 EQU 0F8H        ; 8259 address for A0 = 0
    PIC2 EQU 0FAH        ; 8259 address for A0 = 1
    ICW1 EQU 18H         ; 8259 command word ICW1
    ICW2 EQU 80H         ; 8259 command word ICW2
    ICW4 EQU 03H         ; 8259 command word ICW4
    OCW1 EQU 0FEH        ; 8259 command word OCW1

```

DATA ENDS

STK SEGMENT STACK

```

    BOS DW 64d DUP (?) ; stack depth (bottom of stack)
    TOS LABEL WORD      ; top of stack

```

STK ENDS

```

CODE SEGMENT PUBLIC 'CODE'
ASSUME CS:CODE, DS:DATA, SS:STK

ORG 08000H          ; address in memory for this segment
START:
    MOV AX, DATA
    MOV DS, AX      ; setting data segment address
    MOV AX, STK
    MOV SS, AX      ; setting data segment address
    LEA SP, TOS      ; setting address of SP as top of stack
    CLI              ; clears IF flag
    ; programming the 8255
    <code for programming the 8255>
    ; programming the 8259
    MOV DX, PIC1      ; set I/O address to access ICW1
    MOV AL, ICW1      ; command byte for ICW1
    OUT DX, AL        ; send command byte
    MOV DX, PIC2      ; set I/O address to access ICW2, ICW4 and OCW1
    MOV AL, ICW2      ; command byte for ICW2
    OUT DX, AL        ; send command word

```

```
MOV AL, ICW4      ; command byte for ICW4
OUT DX, AL        ; send command byte
MOV AL, OCW1      ; command byte for OCW1
OUT DX, AL        ; send command word
STI               ; enable INTR pin
; storing interrupt vector in the vector table
<code for storing interrupt vector>
```

HERE:

```
<code for foreground routine>
JMP HERE
CODE ENDS
END START
```

Storing Interrupt Vector to Vector Table

- To store the interrupt vector (from 80H-87H), we need to calculate the address from the given interrupt vector.
- To get the address the interrupt vector must be multiplied by 4 since each vector occupies 4 bytes.

Any interrupt vector

3	Segment (high)
2	Segment (low)
1	Offset (high)
0	Offset (low)

Storing Interrupt Vector to Vector Table

Since the interrupt sources are connected to IR0 and IR2, the interrupt vectors should be 80H and 81H.

absolute address

$$80H \times 4 = \boxed{00200H}$$

offset (IP): 00200H

segment (CS): 00202H

$$81H \times 4 = 00204H$$

offset (IP): 00204H

segment (CS): 00206H

...	
...	
00207H	segment high (81H)
00206H	segment low (81H)
00205H	offset high (81H)
00204H	offset low (81H)
00203H	segment high (80H)
00202H	segment low (80H)
00201H	offset high (80H)
00200H	offset low (80H)
...	
...	

```
MOV AX, OFFSET ISR1    ; get offset address of ISR1 (IP)
MOV [ES:200H], AX      ; store offset address to memory
MOV AX, SEG ISR1       ; get segment address of ISR1 (CS)
MOV [ES:202H], AX      ; store segment address to memory
MOV AX, OFFSET ISR2    ; get offset address of ISR2 (IP)
MOV [ES:204H], AX      ; store offset address to memory
MOV AX, SEG ISR2       ; get segment address of ISR2 (CS)
MOV [ES:206H], AX      ; store segment address to memory
```

ISR1 and ISR2 are the name of the procedure that contains the interrupt service routines for interrupt vectors 80H (IR0) and 81H (IR1).

Interrupt Service Routines (ISR)

- The interrupt service routines (ISR) handles the interrupt request based on their interrupt vectors.
- These could be procedures within the code segment (PROC NEAR) or outside (PROC FAR).
- Each unmasked interrupt vector must have an ISR.


```
PROCED1 SEGMENT
ISR1 PROC FAR
ASSUME CS:PROCED1, DS:DATA
ORG 01000H          ; address in memory for this segment
    PUSHF
    PUSH AX          ; save program context
    PUSH DX
    <ISR code here>
    POP DX           ; retrieve program context
    POP AX
    POPF
    IRET             ; return from interrupt
ISR1 ENDP
PROCED1 ENDS
```

Interrupt service routine ISR1 (80H) outside of the code segment
at memory address 01000H

```
PROCED2 SEGMENT
ISR2 PROC FAR
ASSUME CS:PROCED2, DS:DATA
ORG 02000H          ; address in memory for this segment
    PUSHF
    PUSH AX          ; save program context
    PUSH DX
    <ISR code here>
    POP DX           ; retrieve program context
    POP AX
    POPF
    IRET             ; return from interrupt
ISR2 ENDP
PROCED2 ENDS
```

Interrupt service routine ISR2 (81H) outside of the code segment
at memory address 02000H

Practice Problem

- The foreground routine checks the value of the DIP switch and displays the number at PORTB. Displays '0' if number is greater than 9.
- ISR1 will display '5' at PORTA.
- ISR2 will display '0' at PORTA.



University of San Carlos | Department of
COMPUTER ENGINEERING

CpE 3104 – Microprocessors

End of Lecture

UNIT 8: INTERRUPT INTERFACING

© 2025